

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación · Ingeniería de Software

Programación en Java

Guía del estudiante — fundamentos del lenguaje y la orientación a objetos

Cada listado es un archivo completo que puedes transcribir y ejecutar

Asignatura: Programación (Ingeniería de Software)

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Período: PPA 2026–2027

Versiones de referencia (Java 25 LTS, Maven, JUnit 5) en la sección 2.

Contenido

1	Introducción	4
1.1	¿Qué es Java y por qué aprenderlo?	4
1.2	Cómo funciona Java: la JVM, el JDK y el bytecode	4
1.3	Versiones de Java	5
1.4	Cómo usar esta guía y los niveles de los ejercicios	5
2	Preparación del entorno de desarrollo	5
2.1	Instalar el JDK	5
2.2	El entorno de desarrollo (IntelliJ IDEA)	6
2.3	Compilar y ejecutar	6
2.4	Maven: gestión de dependencias y construcción	7
2.5	Estructura de un proyecto y paquetes	7
2.6	Versiones de referencia (verificadas a junio de 2026)	7
3	Fundamentos del lenguaje	8
3.1	Estructura de una clase y el método main	8
3.2	Variables, tipos primitivos y referencias	8
3.3	Cadenas de texto	9
3.4	Operadores	10
3.5	Estructuras de control	10
3.6	Arreglos	11
3.7	Métodos	12
3.8	Inferencia de tipos con var	12
3.9	Ejercicios — nivel básico	13
4	Programación orientada a objetos	14
4.1	Clases y objetos	14
4.2	Encapsulación	14
4.3	Constructores	15
4.4	Herencia	15
4.5	Polimorfismo y sobreescritura	16
4.6	Clases abstractas e interfaces	17

4.7	Records: modelar datos de forma inmutable	17
4.8	Enumeraciones (enum)	18
4.9	Ejercicios — nivel intermedio	18
5	Colecciones y genéricos	19
5.1	Genéricos: seguridad de tipos	19
5.2	Las colecciones principales: List, Set y Map	19
5.3	Recorrer colecciones	20
5.4	Ordenar y comparar	20
5.5	Ejercicios — colecciones	21
6	Manejo de excepciones	21
6.1	Qué son y por qué importan	21
6.2	try, catch y finally	22
6.3	Excepciones comprobadas y no comprobadas	22
6.4	Crear tus propias excepciones	22
6.5	Ejercicios — excepciones	23
7	Programación funcional moderna	23
7.1	Interfaces funcionales y lambdas	23
7.2	Streams: procesar colecciones declarativamente	24
7.3	Optional: el adiós al null	24
7.4	Ejercicios — programación funcional	25
8	Organización del código y dependencias	25
8.1	Paquetes	25
8.2	Modificadores de acceso y visibilidad	26
8.3	Maven a fondo: dependencias y ciclo de vida	26
8.4	Inyección de dependencias	27
8.5	Ejercicios — organización	27
9	Trabajar con una base de datos (JDBC)	28
9.1	¿Qué es JDBC?	28
9.2	La conexión	28
9.3	Sentencias preparadas	29

9.4	Consultar y recorrer resultados	29
9.5	Transacciones	30
9.6	Ejercicios — base de datos	30
10	Buenas prácticas y herramientas	31
10.1	Convenciones de código	31
10.2	Pruebas con JUnit	31
10.3	Registro de eventos (logging)	31
10.4	Empaquetado en un .jar	32
10.5	Control de versiones	32
11	Proyecto integrador: gestor de estudiantes en consola	32
11.1	Estructura del proyecto	32
11.2	El modelo	32
11.3	El repositorio (acceso a datos)	33
11.4	El servicio (lógica y validación)	34
11.5	El programa principal (menú de consola)	34
11.6	Ejercicio integrador	35
12	Banco de ejercicios por nivel	36
12.1	Nivel básico	36
12.2	Nivel intermedio	36
12.3	Nivel avanzado	36
12.4	Proyectos propuestos	37
13	Soluciones seleccionadas	37
13.1	Número primo (básico 12.1.1)	37
13.2	Frecuencia de palabras (intermedio 12.2.2)	38
13.3	Agrupar con streams (avanzado 12.3.1)	38
14	Glosario de términos	38
15	Recursos y referencias	39

1 Introducción

Aprender a programar bien en Java te da una de las bases más sólidas y demandadas de la ingeniería de software. Java es un lenguaje maduro, robusto y orientado a objetos, con el que se construyen desde pequeñas utilidades hasta sistemas de gran tamaño. Esta guía te lleva paso a paso, desde tu primer programa hasta un proyecto completo, con explicaciones detalladas y ejercicios de todos los niveles.

El recorrido es progresivo: primero el lenguaje y su sintaxis, luego la orientación a objetos —el corazón de Java—, después las colecciones, las excepciones, la programación funcional moderna, la organización del código, el trabajo con datos y, por último, un proyecto integrador. Dominar estos fundamentos es el requisito imprescindible para, más adelante, abordar proyectos de mayor envergadura con Java.

Cada listado es un archivo completo

Todos los ejemplos en Java de esta guía son programas completos. El título de cada cuadro es el nombre del archivo (por ejemplo, `Hola.java`): crea un archivo con ese nombre exacto, transcribe el contenido tal cual y ejecútalo. La clase pública debe llamarse igual que el archivo.

1.1 ¿Qué es Java y por qué aprenderlo?

Java es un lenguaje de programación de propósito general, fuertemente tipado y orientado a objetos, creado por James Gosling en Sun Microsystems y publicado en 1996. Hoy lo mantiene Oracle junto a una enorme comunidad, y es uno de los lenguajes más utilizados del mundo, especialmente en sistemas grandes y de larga vida.

Aprenderlo te aporta tres ventajas duraderas. Primera, su filosofía «escribe una vez, ejecuta en cualquier parte»: el mismo programa corre en Windows, Linux o macOS sin cambios. Segunda, su rigor: el tipado estático y la orientación a objetos te enseñan a diseñar software ordenado y mantenible. Tercera, su ecosistema: una biblioteca estándar inmensa y herramientas maduras que usarás durante toda tu carrera.

1.2 Cómo funciona Java: la JVM, el JDK y el bytecode

Java no se compila directamente a instrucciones del procesador, sino a un formato intermedio llamado bytecode. Ese bytecode lo ejecuta la JVM (Java Virtual Machine), una «máquina» que existe para cada sistema operativo. Por eso un mismo programa funciona en todas partes: cada JVM traduce el bytecode al hardware concreto.

Para programar necesitas el JDK (Java Development Kit), que incluye el compilador (`javac`), la máquina virtual (`java`) y la biblioteca estándar. El compilador transforma tu código fuente (`.java`) en bytecode (`.class`), y la JVM lo ejecuta. Entender esta cadena —fuente, compilación, bytecode, ejecución— te ayudará a interpretar los errores y a configurar bien tus proyectos.

1.3 Versiones de Java

Java publica una versión nueva cada seis meses y, cada dos años, una versión de soporte prolongado (LTS, Long-Term Support) pensada para proyectos serios. Conviene trabajar siempre sobre una versión LTS por estabilidad y soporte.

A mediados de 2026, la LTS vigente es Java 25 (septiembre de 2025) y la anterior es Java 21; la versión más reciente por funcionalidad es Java 26 (marzo de 2026, no LTS). El código de esta guía usa características disponibles desde Java 21 —como los **record**, las expresiones **switch** y los bloques de texto— y funciona igual en Java 25.

1.4 Cómo usar esta guía y los niveles de los ejercicios

Lee cada sección, reproduce los ejemplos en tu propio entorno y resuelve los ejercicios: programar se aprende programando. Teclea el código en lugar de copiarlo; así descubres detalles que de otro modo pasan inadvertidos y aprendes a leer los mensajes del compilador.

Los ejercicios están etiquetados por nivel para que dosifiques tu práctica:

- **[Básico]** afianzan la sintaxis y los conceptos esenciales; deberías resolverlos justo tras leer la sección.
- **[Intermedio]** combinan varios conceptos y se acercan a problemas reales (objetos, colecciones, excepciones).
- **[Avanzado]** exigen diseño, genéricos, programación funcional y trabajo con datos; son los más cercanos a un proyecto real.

Al final encontrarás un banco de ejercicios por nivel y un conjunto de soluciones comentadas para comparar tu enfoque con uno posible.

2 Preparación del entorno de desarrollo

Antes de escribir código necesitas instalar el JDK y un entorno cómodo para editar, compilar y ejecutar. Esta sección te guía en la instalación y te muestra cómo organizar un proyecto desde el principio con buenas prácticas.

2.1 Instalar el JDK

El JDK es lo único imprescindible para empezar. Puedes usar la distribución de Oracle o una abierta como Eclipse Temurin (OpenJDK); ambas son equivalentes para aprender. Instala una versión LTS (21 o 25) y, al terminar, comprueba en una terminal que quedó disponible.

Verificar el JDK

```
1 C:\> java -version
2 openjdk version "25" 2025-09-16
3 OpenJDK Runtime Environment (build 25+...)
4
5 C:\> javac -version
6 javac 25
```

La variable JAVA_HOME

Muchas herramientas (como Maven) necesitan la variable de entorno `JAVA_HOME` apuntando a la carpeta del JDK. Si una herramienta no encuentra Java, revisa que esté bien definida.

2.2 El entorno de desarrollo (IntelliJ IDEA)

Aunque puedes programar con un editor simple, un IDE te ahorra muchísimo tiempo. IntelliJ IDEA es el más usado para Java: ofrece autocompletado, análisis de errores en vivo, refactorización y un depurador potente. Al crear un proyecto, seleccionas el JDK instalado y eliges si gestionarlo con Maven.

Familiarízate pronto con dos acciones clave del IDE: ejecutar (el botón con forma de triángulo verde) y depurar (ejecutar paso a paso para inspeccionar el valor de las variables). Saber depurar es una de las habilidades que más te distinguirá como programador.

2.3 Compilar y ejecutar

Conviene entender qué hace el IDE por debajo. Un programa Java se compila con `javac` y se ejecuta con `java`. El compilador genera un archivo `.class` con el bytecode, que la máquina virtual ejecuta. Desde Java 11 también puedes ejecutar un archivo fuente directamente con `java Archivo.java`, muy cómodo para los ejemplos de esta guía.

Compilar y ejecutar

```
1 C:\> javac Hola.java          # genera Hola.class
2 C:\> java Hola                # ejecuta el programa
3 Hola, mundo
4
5 C:\> java Hola.java           # alternativa: compila y ejecuta de una vez
6 Hola, mundo
```

En el IDE no harás esto manualmente, pero saber que ocurre te ayudará a entender los errores de compilación (los detecta `javac`) frente a los errores en ejecución (los lanza la JVM mientras corre el programa).

2.4 Maven: gestión de dependencias y construcción

Casi ningún proyecto vive aislado: usa bibliotecas de terceros. Maven es la herramienta estándar para gestionar esas dependencias y para construir el proyecto de forma reproducible. Se configura con un archivo `pom.xml` que declara qué bibliotecas necesitas; Maven las descarga y las pone a disposición de tu código.

pom.xml

```
1 <project>
2   <modelVersion>4.0.0</modelVersion>
3   <groupId>com.uteq</groupId>
4   <artifactId>practica</artifactId>
5   <version>1.0.0</version>
6   <properties>
7     <maven.compiler.release>21</maven.compiler.release>
8     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
9   </properties>
10 </project>
```

Más adelante añadirás dependencias dentro de `<dependencies>`. Con Maven, compartir un proyecto es trivial: quien lo reciba ejecuta un comando y obtiene exactamente las mismas bibliotecas.

2.5 Estructura de un proyecto y paquetes

Un proyecto Maven sigue una estructura de carpetas convenida que todo el mundo reconoce. El código fuente va en `src/main/java`, organizado en paquetes (carpetas con un nombre jerárquico, como `com.uteq.modelo`). Los paquetes evitan choques de nombres y ordenan el código por responsabilidades.

Estructura típica

```
1 practica/
2 |- pom.xml
3 |- src/
4   |- main/java/com/uteq/
5     |- Main.java
6     |- modelo/Estudiante.java
7     |- repositorio/EstudianteRepositorio.java
8     |- servicio/EstudianteServicio.java
9   |- test/java/com/uteq/      # pruebas
```

2.6 Versiones de referencia (verificadas a junio de 2026)

Estas son las versiones vigentes de las herramientas usadas en la guía. Las dependencias se gestionan con Maven, que descarga la versión que declares; no es necesario fijarlas a mano fuera del `pom.xml`.

Componente	Versión vigente (jun. 2026)
Java (LTS)	25 (actual) y 21 (anterior); recomendadas para proyectos.
Java (última)	26 (marzo de 2026, no LTS).
Maven	3.9+ (gestor de dependencias y construcción).
IntelliJ IDEA	2026.1.x (Community o Ultimate).
JUnit	5 (Jupiter), para pruebas.
PostgreSQL	18.4 (para la sección de persistencia).

3 Fundamentos del lenguaje

En esta sección recorrerás la base de Java: cómo se escribe un programa, cómo se declaran datos, cómo se controla el flujo y cómo se agrupan instrucciones en métodos. Son los cimientos sobre los que se apoya todo lo demás, así que dedícales tiempo y resuelve los ejercicios del final.

3.1 Estructura de una clase y el método main

En Java, todo el código vive dentro de clases. Un programa ejecutable necesita un método especial llamado **main**, que es por donde empieza la ejecución. El siguiente archivo es el clásico primer programa: muestra un texto por la consola. Recuerda: la clase pública debe llamarse igual que el archivo.

Hola.java

```
1 public class Hola {
2     public static void main(String[] args) {
3         // System.out.println imprime y salta de línea
4         System.out.println("Hola, mundo");
5     }
6 }
```

El método **main** es siempre `public static void main(String[] args)`: la JVM lo busca con esa firma exacta para arrancar. Los comentarios se escriben con `//` (una línea) o `/* ... */` (varias).

3.2 Variables, tipos primitivos y referencias

Java es de tipado estático: cada variable tiene un tipo fijo que declaras y que no cambia. Hay dos familias de tipos. Los primitivos guardan valores simples (números, caracteres, booleanos) directamente. Los tipos de referencia (como **String** y los objetos) guardan una referencia al dato.

Variables.java

```

1 public class Variables {
2     public static void main(String[] args) {
3         int edad = 21;           // entero
4         double nota = 8.5;       // decimal
5         char inicial = 'A';      // un caracter
6         boolean activo = true;   // verdadero o falso
7         long grande = 9000000000L;
8         String nombre = "Ana";   // tipo de referencia
9         final double IVA = 0.15; // 'final' = constante
10
11         System.out.println(nombre + " tiene " + edad);
12         System.out.println("Nota: " + nota + " IVA: " + IVA);
13         System.out.println(inicial + " " + activo + " " + grande);
14     }
15 }

```

Usa `final` para los valores que no deben cambiar: el compilador te protegerá si intentas reasignarlos, y el código se vuelve más claro. Como norma, declara las variables lo más cerca posible de donde se usan.

3.3 Cadenas de texto

La clase `String` representa texto y es inmutable: cada operación que «modifica» una cadena en realidad crea una nueva. Para construir texto en bucles conviene usar `StringBuilder`, que sí es mutable. Desde Java 15 existen además los bloques de texto para escribir varias líneas con comodidad.

Cadenas.java

```

1 public class Cadenas {
2     public static void main(String[] args) {
3         String nombre = "Ana";
4         String saludo = "Hola, " + nombre; // concatenacion con +
5         System.out.println(saludo);
6         System.out.println(saludo.length()); // longitud
7         System.out.println(saludo.toUpperCase());
8
9         // En Java las cadenas NO interpolan variables:
10        String clave = "P@$$Gr3$QL"; // literal, sin precaucion
11        System.out.println(clave);
12
13        String bloque = ""
14            Primera linea
15            Segunda linea""; // bloque de texto (Java 15+)
16        System.out.println(bloque);
17    }
18 }

```

Comparar cadenas

Para comparar el contenido de dos `String` usa `a.equals(b)`, NUNCA `a == b`. El operador `==` compara referencias (si son el mismo objeto), no el texto: es una de las confusiones más frecuentes al empezar.

3.4 Operadores

Los operadores combinan valores. Además de los aritméticos y lógicos habituales, conviene conocer los de comparación y el operador ternario, que resume un `if/else` sencillo en una sola expresión.

Operador	Significado y ejemplo
<code>+ - * / %</code>	Aritméticos; <code>%</code> es el resto de la división.
<code>== !=</code>	Igualdad y desigualdad (en primitivos).
<code>< > <= >=</code>	Comparaciones de orden.
<code>&& !</code>	Y, O, NO lógicos (con cortocircuito).
<code>? :</code>	Ternario: <code>condicion ? valorSi : valorNo</code> .
<code>+= -= ++ --</code>	Asignación compuesta e incremento/decremento.

Recuerda la distinción de la sección anterior: con objetos (incluido `String`), `==` compara referencias; para comparar contenido se usa `equals`.

3.5 Estructuras de control

Las estructuras de control deciden qué se ejecuta y cuántas veces. Java ofrece las clásicas (`if`, `for`, `while`) y, en versiones modernas, la expresión `switch`, que devuelve un valor y es más segura y concisa que el `switch` tradicional. El siguiente archivo combina un condicional con una expresión `switch`.

Control.java

```

1 public class Control {
2     public static void main(String[] args) {
3         int nota = 8;
4         if (nota >= 7) {
5             System.out.println("Aprobado");
6         } else {
7             System.out.println("Reprobado");
8         }
9         String letra = switch (nota) {    // expresion switch (Java 14+)
10             case 9, 10 -> "A";
11             case 7, 8 -> "B";
12             default -> "C";
13         };
14         System.out.println(letra);
15     }
16 }

```

El siguiente archivo muestra los dos bucles que más usarás: el **for** clásico (cuando necesitas un índice) y el **for-each** (para recorrer arreglos y colecciones de forma idiomática).

Bucles.java

```

1 public class Bucles {
2     public static void main(String[] args) {
3         for (int i = 1; i <= 3; i++) {
4             System.out.print(i);           // 123
5         }
6         System.out.println();
7         String[] colores = {"rojo", "verde", "azul"};
8         for (String c : colores) {        // for-each
9             System.out.print(c + " ");
10        }
11        System.out.println();
12    }
13 }

```

3.6 Arreglos

Un arreglo guarda un número fijo de elementos del mismo tipo, accesibles por un índice que empieza en cero. Es la estructura más básica para agrupar datos; más adelante verás las colecciones, que son más flexibles.

Arreglos.java

```

1 public class Arreglos {
2     public static void main(String[] args) {
3         int[] numeros = {10, 20, 30};
4         System.out.println(numeros[0]);    // 10
5         System.out.println(numeros.length); // 3
6         numeros[1] = 99;                  // modificar
7         int suma = 0;
8         for (int n : numeros) suma += n;
9         System.out.println("Suma: " + suma);
10    }
11 }

```

3.7 Métodos

Un método agrupa instrucciones reutilizables; recibe parámetros (con su tipo) y puede devolver un valor (o `void` si no devuelve nada). Java permite la sobrecarga: varios métodos con el mismo nombre pero distinta lista de parámetros. El siguiente archivo define y prueba tres métodos.

Metodos.java

```

1 public class Metodos {
2     static int sumar(int a, int b) { return a + b; }
3     static double sumar(double a, double b) { return a + b; } // sobrecarga
4     static int sumarTodos(int... valores) {                  // varargs
5         int total = 0;
6         for (int v : valores) total += v;
7         return total;
8     }
9     public static void main(String[] args) {
10        System.out.println(sumar(2, 3));    // 5
11        System.out.println(sumar(2.5, 3.5)); // 6.0
12        System.out.println(sumarTodos(1, 2, 3, 4)); // 10
13    }
14 }

```

Un buen método hace una sola cosa y la hace bien. Si crece demasiado o necesita muchos comentarios para entenderse, suele convenir dividirlo en métodos más pequeños.

3.8 Inferencia de tipos con `var`

Desde Java 10, dentro de un método puedes declarar variables locales con `var`: el compilador deduce el tipo a partir del valor asignado. No es tipado dinámico —el tipo sigue siendo fijo—, solo te ahorra escribirlo cuando es evidente. Úsalo con criterio, sin sacrificar la claridad.

VarDemo.java

```

1 public class VarDemo {
2     public static void main(String[] args) {
3         var nombre = "Ana";           // el compilador deduce: String
4         var edad   = 21;               // deduce: int
5         var notas  = new int[]{7, 8, 9};
6         System.out.println(nombre + " " + edad + " " + notas.length);
7     }
8 }

```

3.9 Ejercicios — nivel básico

Estos ejercicios consolidan la sintaxis que acabas de ver. Resuélvelos en archivos con su clase y su método `main`, y ejecútalos para comprobar el resultado. Intenta no mirar los ejemplos mientras los resuelves.

Pon en práctica los fundamentos:

1. **[Básico]** Escribe un programa que muestre tu nombre y tu edad en dos líneas.
2. **[Básico]** Declara dos números y muestra su suma, resta, producto y resto (%).
3. **[Básico]** Pide (con valores fijos por ahora) una nota y muestra «Aprobado» o «Reprobado» según sea mayor o igual a 7.
4. **[Básico]** Usa una expresión `switch` para convertir un número del 1 al 7 en el día de la semana.
5. **[Básico]** Dado un arreglo de cinco notas, calcula y muestra el promedio.
6. **[Básico]** Escribe un método `esPar(int n)` que devuelva `boolean` y pruébalo.
7. **[Básico]** Muestra la tabla de multiplicar de un número con un bucle `for`.
8. **[Básico]** Invierte el contenido de un arreglo mostrándolo del último al primer elemento.

Como referencia, este archivo resuelve el ejercicio del promedio y es ejecutable tal cual:

Promedio.java

```

1 public class Promedio {
2     public static void main(String[] args) {
3         double[] notas = {7, 8.5, 6, 9, 10};
4         double suma = 0;
5         for (double n : notas) suma += n;
6         System.out.println("Promedio: " + (suma / notas.length));
7     }
8 }

```

4 Programación orientada a objetos

La orientación a objetos es el corazón de Java y la forma en que se diseñan los programas grandes. Consiste en modelar el problema con «objetos» que combinan datos (atributos) y comportamiento (métodos). Esta es la sección más importante de la guía: tómate tu tiempo y practica cada concepto.

4.1 Clases y objetos

Una clase es un molde que define qué atributos y qué métodos tendrán sus objetos. Un objeto es una instancia concreta de una clase, creada con **new**. Dentro de los métodos, la palabra **this** se refiere al objeto actual. En un mismo archivo puede haber una clase pública (con el **main**) y otras clases de apoyo.

CuentaDemo.java

```
1 public class CuentaDemo {
2     public static void main(String[] args) {
3         Cuenta c = new Cuenta();
4         c.depositar(100);
5         System.out.println(c.saldo);    // 100.0
6     }
7 }
8
9 class Cuenta {
10     double saldo;                // atributo
11     void depositar(double monto) { // método
12         this.saldo += monto;
13     }
14 }
```

4.2 Encapsulación

La encapsulación consiste en ocultar los detalles internos de un objeto y exponer solo lo necesario. En la práctica, los atributos se declaran **private** y se accede a ellos mediante métodos **public** (los «getters» y «setters»). Así controlas cómo se leen y modifican los datos.

EncapsulacionDemo.java

```

1 public class EncapsulacionDemo {
2     public static void main(String[] args) {
3         Cuenta c = new Cuenta();
4         c.depositar(100);
5         c.depositar(-50);           // ignorado por la validacion
6         System.out.println(c.getSaldo()); // 100.0
7     }
8 }
9
10 class Cuenta {
11     private double saldo;           // oculto al exterior
12     public void depositar(double monto) {
13         if (monto > 0) this.saldo += monto; // validacion
14     }
15     public double getSaldo() { return this.saldo; }
16 }

```

La ventaja es el control: como nadie puede tocar `saldo` directamente, el método `depositar` puede validar (aquí, rechazar montos negativos). La encapsulación es la primera línea de defensa de la integridad de tus datos.

4.3 Constructores

El constructor es un método especial que se ejecuta al crear el objeto con `new` y sirve para inicializarlo. Tiene el mismo nombre que la clase y no declara tipo de retorno. Si no escribes ninguno, Java provee uno vacío por defecto.

ConstructorDemo.java

```

1 public class ConstructorDemo {
2     public static void main(String[] args) {
3         Estudiante e = new Estudiante("Ana", "ana@uteq.edu.ec");
4         System.out.println(e.getNombre());
5     }
6 }
7
8 class Estudiante {
9     private String nombre;
10    private String correo;
11    public Estudiante(String nombre, String correo) {
12        this.nombre = nombre;
13        this.correo = correo;
14    }
15    public String getNombre() { return nombre; }
16 }

```

4.4 Herencia

La herencia permite crear una clase a partir de otra, reutilizando y extendiendo su comportamiento. La subclase usa `extends` y hereda los atributos y métodos de la superclase; con `super` puede invocar

al constructor o a métodos de la clase padre.

HerenciaDemo.java

```

1 public class HerenciaDemo {
2     public static void main(String[] args) {
3         Perro p = new Perro("Fido");
4         System.out.println(p.nombre + ": " + p.sonido());
5     }
6 }
7
8 class Animal {
9     protected String nombre;
10    public Animal(String nombre) { this.nombre = nombre; }
11    public String sonido() { return "..."; }
12 }
13
14 class Perro extends Animal {
15     public Perro(String nombre) { super(nombre); }
16     @Override public String sonido() { return "Guau"; }
17 }

```

4.5 Polimorfismo y sobreescritura

El polimorfismo permite tratar objetos de distintas subclases a través de su superclase común, y que cada uno responda a su manera. La sobreescritura (anotada con `@Override`) es redefinir en la subclase un método del padre. Es uno de los mecanismos más potentes de la orientación a objetos.

PolimorfismoDemo.java

```

1 public class PolimorfismoDemo {
2     public static void main(String[] args) {
3         Animal[] animales = { new Perro("Fido"), new Animal("X") };
4         for (Animal a : animales) {
5             System.out.println(a.sonido());    // cada uno responde distinto
6         }
7     }
8 }
9
10 class Animal {
11     protected String nombre;
12     public Animal(String nombre) { this.nombre = nombre; }
13     public String sonido() { return "..."; }
14 }
15
16 class Perro extends Animal {
17     public Perro(String nombre) { super(nombre); }
18     @Override public String sonido() { return "Guau"; }
19 }

```

La anotación `@Override` no es obligatoria, pero úsala siempre: le pide al compilador que verifique que realmente estás sobreescribiendo un método, y te avisa si te equivocas en la firma.

4.6 Clases abstractas e interfaces

A veces quieres definir un concepto general sin implementarlo del todo. Una clase abstracta (**abstract**) puede tener métodos sin cuerpo que las subclasses deben completar. Una interfaz (**interface**) va más allá: define un contrato de métodos que cualquier clase puede implementar con **implements**. Las interfaces son clave para escribir código flexible.

FigurasDemo.java

```

1 public class FigurasDemo {
2     public static void main(String[] args) {
3         Figura[] figuras = { new Circulo(2), new Cuadrado(3) };
4         for (Figura f : figuras) {
5             System.out.println(f.area());    // polimorfismo
6         }
7     }
8 }
9
10 interface Figura { double area(); }
11
12 class Circulo implements Figura {
13     private double radio;
14     public Circulo(double radio) { this.radio = radio; }
15     @Override public double area() { return Math.PI * radio * radio; }
16 }
17
18 class Cuadrado implements Figura {
19     private double lado;
20     public Cuadrado(double lado) { this.lado = lado; }
21     @Override public double area() { return lado * lado; }
22 }

```

Programar contra interfaces, y no contra clases concretas, es una de las claves del software mantenible: puedes cambiar la implementación sin tocar el código que la usa. Lo verás en acción en la sección de organización del código.

4.7 Records: modelar datos de forma inmutable

Desde Java 16, un **record** es una forma muy concisa de crear clases cuyo único propósito es transportar datos. Con una sola línea obtienes los atributos, el constructor, los métodos de acceso y las comparaciones, todo inmutable. Son ideales para representar entidades simples.

RecordDemo.java

```

1 public class RecordDemo {
2     // Un record equivale a una clase con constructor, getters, equals y toString:
3     record Estudiante(int id, String nombre, String correo) {}
4
5     public static void main(String[] args) {
6         var e = new Estudiante(1, "Ana", "ana@uteq.edu.ec");
7         System.out.println(e.nombre());    // metodo de acceso: Ana
8         System.out.println(e);             // toString automatico
9     }
10 }

```

4.8 Enumeraciones (enum)

Una enumeración representa un conjunto fijo de valores posibles, como los estados de una cuenta o los roles de una persona. Usar un `enum` en lugar de constantes sueltas hace que el compilador garantice que solo se usen valores válidos.

EnumDemo.java

```

1 public class EnumDemo {
2     enum Estado { ACTIVO, INACTIVO, SUSPENDIDO }
3
4     public static void main(String[] args) {
5         Estado e = Estado.ACTIVO;
6         String mensaje = switch (e) {
7             case ACTIVO    -> "La cuenta esta activa";
8             case INACTIVO  -> "La cuenta esta inactiva";
9             case SUSPENDIDO -> "La cuenta esta suspendida";
10        };
11        System.out.println(mensaje);
12    }
13 }

```

4.9 Ejercicios — nivel intermedio

Estos ejercicios te hacen diseñar con objetos, no solo escribir instrucciones sueltas. Es el salto mental más importante del curso. Recuerda que cada solución es un archivo completo con su `main`.

Modela con clases:

1. **[Intermedio]** Crea una clase `Producto` con nombre y precio encapsulados, constructor y un método que devuelva el precio con IVA.
2. **[Intermedio]** Define una interfaz `Notificable` con `enviar(String mensaje)` e impleméntala en dos clases distintas.
3. **[Intermedio]** Modela una jerarquía `Figura` (interfaz) con `Circulo`, `Rectangulo` y `Triangulo`; guarda varias en un arreglo y recórrelas mostrando su área (polimorfismo).

4. **[Intermedio]** Crea un `record Punto(int x, int y)` y un método que calcule la distancia entre dos puntos.
5. **[Intermedio]** Define un `enum Rol (ADMIN, DOCENTE, ESTUDIANTE)` y un método que devuelva los permisos de cada rol con `switch`.
6. **[Avanzado]** Diseña una clase `Cuenta` con depósito, retiro (que no permita saldo negativo) y un historial de operaciones.

5 Colecciones y genéricos

Los arreglos tienen un tamaño fijo; en la práctica necesitas estructuras que crezcan y que ofrezcan operaciones listas para usar. Para eso está el marco de colecciones de Java, que se apoya en los genéricos para garantizar la seguridad de tipos. Esta sección es esencial para casi cualquier programa real.

5.1 Genéricos: seguridad de tipos

Los genéricos permiten escribir clases y métodos que funcionan con cualquier tipo, indicándolo entre ángulos `<>`. Su gran ventaja es que el compilador verifica los tipos: una lista de `String` no aceptará un número por error. Verás los genéricos sobre todo al usar colecciones.

GenericosDemo.java

```
1 import java.util.*;
2
3 public class GenericosDemo {
4     public static void main(String[] args) {
5         List<String> nombres = new ArrayList<>();    // solo String
6         nombres.add("Ana");
7         // nombres.add(42);    // ERROR en compilacion: no es String
8         String primero = nombres.get(0);           // sin conversiones
9         System.out.println(primero);
10    }
11 }
```

5.2 Las colecciones principales: List, Set y Map

El marco de colecciones ofrece tres familias fundamentales. `List` es una secuencia ordenada que admite duplicados; `Set` es un conjunto sin duplicados; y `Map` asocia claves con valores (como un diccionario). Elige según lo que necesites representar.

ColeccionesDemo.java

```

1 import java.util.*;
2
3 public class ColeccionesDemo {
4     public static void main(String[] args) {
5         List<String> lista = new ArrayList<>(List.of("a", "b", "a"));
6         System.out.println(lista.size());    // 3 (admite duplicados)
7
8         Set<String> conjunto = new HashSet<>(lista);
9         System.out.println(conjunto.size()); // 2 (sin duplicados)
10
11         Map<String, Integer> edades = new HashMap<>();
12         edades.put("Ana", 21);
13         edades.put("Luis", 23);
14         System.out.println(edades.get("Ana")); // 21
15     }
16 }

```

5.3 Recorrer colecciones

La forma habitual de recorrer una colección es el bucle **for-each**. Para los mapas, se recorren sus entradas (pares clave-valor). Conocer estas formas te permite mostrar y procesar datos con soltura.

IteracionDemo.java

```

1 import java.util.*;
2
3 public class IteracionDemo {
4     public static void main(String[] args) {
5         List<String> lista = List.of("a", "b", "c");
6         for (String s : lista) {
7             System.out.println(s);
8         }
9         Map<String, Integer> edades = Map.of("Ana", 21, "Luis", 23);
10        for (Map.Entry<String, Integer> e : edades.entrySet()) {
11            System.out.println(e.getKey() + " -> " + e.getValue());
12        }
13    }
14 }

```

5.4 Ordenar y comparar

Para ordenar objetos, Java necesita saber cómo compararlos. La interfaz **Comparable** define el orden «natural» de una clase, y **Comparator** permite definir órdenes alternativos sin tocar la clase. Ambas son muy útiles al presentar datos ordenados.

OrdenarDemo.java

```

1 import java.util.*;
2
3 public class OrdenarDemo {
4     public static void main(String[] args) {
5         List<String> nombres = new ArrayList<>(List.of("Luis", "Ana", "Eva"));
6         Collections.sort(nombres);                // alfabetico
7         System.out.println(nombres);
8         nombres.sort(Comparator.reverseOrder());    // inverso
9         System.out.println(nombres);
10        nombres.sort(Comparator.comparingInt(String::length)); // por longitud
11        System.out.println(nombres);
12    }
13 }

```

5.5 Ejercicios — colecciones

Estos ejercicios te hacen elegir la estructura adecuada y operar con ella.

Trabaja con colecciones:

1. **[Intermedio]** Guarda una lista de nombres, elimina los duplicados pasándola a un **Set** y muéstrala ordenada.
2. **[Intermedio]** Usa un **Map** para contar cuántas veces aparece cada palabra en una frase.
3. **[Intermedio]** Dada una lista de objetos **Estudiante**, ordénala por nombre con un **Comparator**.
4. **[Avanzado]** Implementa una agenda (**Map** de nombre a teléfono) con alta, baja y búsqueda.
5. **[Avanzado]** Escribe un método genérico `<T> T primero(List<T> lista)` que devuelva el primer elemento o lance una excepción si está vacía.

6 Manejo de excepciones

Los programas fallan: un dato es inválido, un archivo no existe, una conversión no es posible. Java gestiona estos casos con excepciones, un mecanismo que separa el flujo normal del manejo de errores y evita que el programa termine de forma abrupta. Manejarlas bien es señal de código profesional.

6.1 Qué son y por qué importan

Una excepción es un objeto que representa un error ocurrido durante la ejecución. Cuando algo va mal, se «lanza» una excepción que interrumpe el flujo normal; si no se captura, el programa termina mostrando un rastro del error. Capturarlas te permite reaccionar de forma controlada.

6.2 try, catch y finally

El bloque **try** contiene el código que podría fallar; el **catch** captura la excepción y reacciona; el **finally** (opcional) se ejecuta siempre, haya error o no. El siguiente archivo provoca y captura una división por cero.

ExcepcionesDemo.java

```

1 public class ExcepcionesDemo {
2     public static void main(String[] args) {
3         try {
4             int resultado = 10 / 0;           // lanza ArithmeticException
5             System.out.println(resultado);
6         } catch (ArithmeticException e) {
7             System.out.println("Error: " + e.getMessage());
8         } finally {
9             System.out.println("Esto se ejecuta siempre");
10        }
11    }
12 }

```

6.3 Excepciones comprobadas y no comprobadas

Java distingue dos tipos. Las no comprobadas (subclases de `RuntimeException`, como dividir por cero) suelen indicar errores de programación. Las comprobadas obligan a declararlas con **throws** o a capturarlas: el compilador te fuerza a tenerlas en cuenta. El siguiente archivo lanza y captura una excepción al validar.

ThrowsDemo.java

```

1 public class ThrowsDemo {
2     static int dividir(int a, int b) {
3         if (b == 0) {
4             throw new IllegalArgumentException("No se divide por cero");
5         }
6         return a / b;
7     }
8     public static void main(String[] args) {
9         System.out.println(dividir(10, 2));    // 5
10        try {
11            dividir(10, 0);
12        } catch (IllegalArgumentException e) {
13            System.out.println("Capturado: " + e.getMessage());
14        }
15    }
16 }

```

6.4 Crear tus propias excepciones

Cuando los errores de tu programa tienen un significado propio, conviene crear excepciones específicas extendiendo `Exception` (comprobada) o `RuntimeException` (no comprobada). Un nombre

descriptivo hace el código mucho más legible y el manejo más preciso.

ExcepcionPropiaDemo.java

```

1 public class ExcepcionPropiaDemo {
2     public static void main(String[] args) {
3         double saldo = 100, monto = 150;
4         try {
5             if (monto > saldo) {
6                 throw new SaldoInsuficienteException("Fondos insuficientes");
7             }
8         } catch (SaldoInsuficienteException e) {
9             System.out.println(e.getMessage());
10        }
11    }
12 }
13
14 class SaldoInsuficienteException extends RuntimeException {
15     public SaldoInsuficienteException(String mensaje) { super(mensaje); }
16 }

```

6.5 Ejercicios — excepciones

Estos ejercicios te hacen anticipar y manejar los fallos con elegancia.

Maneja los errores:

1. **[Intermedio]** Escribe un método que convierta un texto a número y capture la excepción si el texto no es válido, devolviendo 0.
2. **[Intermedio]** Crea una excepción `EdadInvalidaException` y lánzala si una edad es negativa.
3. **[Avanzado]** Implementa un método de retiro en una `Cuenta` que lance `SaldoInsuficienteException` y captúrala donde lo llamas.
4. **[Avanzado]** Usa `try-with-resources` para leer un recurso y garantizar que se cierra siempre.

7 Programación funcional moderna

Desde Java 8, el lenguaje incorpora ideas de la programación funcional que hacen el código más expresivo y conciso, sobre todo al procesar colecciones. No reemplazan a la orientación a objetos: la complementan. Verás lambdas, flujos (streams) y el manejo seguro de valores ausentes.

7.1 Interfaces funcionales y lambdas

Una interfaz funcional es una interfaz con un único método. Una expresión lambda es una forma breve de implementarla sin escribir una clase entera: es, en esencia, una función anónima. Las lambdas son la base de todo el estilo funcional de Java.

LambdasDemo.java

```

1 import java.util.function.*;
2
3 public class LambdasDemo {
4     public static void main(String[] args) {
5         Function<Integer, Integer> doble = x -> x * 2;
6         System.out.println(doble.apply(5));           // 10
7         Predicate<Integer> esPar = n -> n % 2 == 0;
8         System.out.println(esPar.test(4));           // true
9     }
10 }

```

7.2 Streams: procesar colecciones declarativamente

Un stream (flujo) permite procesar una colección describiendo QUÉ quieres hacer (filtrar, transformar, agrupar) en lugar de CÓMO recorrerla. Encadenas operaciones como `filter`, `map` y `collect`, y el resultado es un código corto y muy legible.

StreamsDemo.java

```

1 import java.util.*;
2 import java.util.stream.*;
3
4 public class StreamsDemo {
5     public static void main(String[] args) {
6         List<Integer> numeros = List.of(1, 2, 3, 4, 5, 6);
7         List<Integer> paresAlCuadrado = numeros.stream()
8             .filter(n -> n % 2 == 0)           // quedarse con los pares
9             .map(n -> n * n)                   // elevar al cuadrado
10            .collect(Collectors.toList());
11         System.out.println(paresAlCuadrado);   // [4, 16, 36]
12
13         int suma = numeros.stream().mapToInt(Integer::intValue).sum();
14         System.out.println(suma);              // 21
15     }
16 }

```

7.3 Optional: el adiós al null

Muchos errores nacen de valores `null` inesperados. `Optional` es un envoltorio que representa explícitamente «puede haber un valor o no», obligándote a considerar el caso vacío. Es la forma moderna de devolver resultados que podrían faltar.

OptionalDemo.java

```

1 import java.util.*;
2
3 public class OptionalDemo {
4     public static void main(String[] args) {
5         Optional<String> resultado = Optional.of("Ana");
6         System.out.println(resultado.orElse("desconocido"));    // Ana
7         Optional<String> vacio = Optional.empty();
8         System.out.println(vacio.orElse("desconocido"));        // desconocido
9     }
10 }

```

7.4 Ejercicios — programación funcional

Estos ejercicios te hacen pensar en términos de transformaciones de datos.

Piensa en flujos:

1. **[Intermedio]** Dada una lista de nombres, usa un stream para quedarte con los que empiezan por «A» y mostrarlos en mayúsculas.
2. **[Intermedio]** Calcula el promedio de una lista de notas con streams.
3. **[Avanzado]** A partir de una lista de `Estudiante`, agrúpalos por carrera usando `Collectors.groupingBy`.
4. **[Avanzado]** Escribe un método que devuelva un `Optional` al buscar por id en una lista.

8 Organización del código y dependencias

A medida que un programa crece, su estructura importa tanto como su funcionamiento. En esta sección verás cómo organizar el código en paquetes, controlar la visibilidad, gestionar dependencias con Maven y aplicar una técnica de diseño —la inyección de dependencias— que prepara el terreno para proyectos grandes.

8.1 Paquetes

Un paquete agrupa clases relacionadas bajo un nombre jerárquico (por convención, en minúsculas y con tu dominio invertido, como `com.uteq.modelo`). La primera línea de cada archivo declara su paquete, y `import` trae clases de otros paquetes. Estos dos archivos, colocados en sus carpetas, forman un proyecto que se ejecuta con Maven.

src/main/java/com/uteq/modelo/Estudiante.java

```

1 package com.uteq.modelo;
2
3 public record Estudiante(int id, String nombre) {}

```

src/main/java/com/uteq/Main.java

```

1 package com.uteq;
2
3 import com.uteq.modelo.Estudiante;
4
5 public class Main {
6     public static void main(String[] args) {
7         var e = new Estudiante(1, "Ana");
8         System.out.println(e);
9     }
10 }

```

8.2 Modificadores de acceso y visibilidad

Java controla quién puede ver cada clase, atributo o método con cuatro niveles de visibilidad. Usarlos bien refuerza la encapsulación y hace explícito qué forma parte de la interfaz pública y qué es detalle interno.

Modificador	Quién puede acceder
<code>public</code>	Cualquier clase, desde cualquier paquete.
<code>protected</code>	La misma clase, su paquete y las subclases.
<code>(sin modificador)</code>	Solo dentro del mismo paquete (acceso de paquete).
<code>private</code>	Solo dentro de la misma clase.

8.3 Maven a fondo: dependencias y ciclo de vida

Ya conoces el `pom.xml`; ahora le añadirás dependencias. Cada una se identifica por tres datos: grupo, artefacto y versión. Maven las descarga de un repositorio central y las pone en tu proyecto. Además define un ciclo de vida con fases como compilar, probar y empaquetar.

Añadir una dependencia (en el pom.xml)

```

1 <dependencies>
2   <dependency>
3     <groupId>org.postgresql</groupId>
4     <artifactId>postgresql</artifactId>
5     <version>42.7.4</version>
6   </dependency>
7 </dependencies>

```

Comandos habituales de Maven

```

1 mvn compile      # compila el código fuente
2 mvn test         # ejecuta las pruebas
3 mvn package      # genera el .jar en target/
4 mvn clean        # borra lo generado

```

8.4 Inyección de dependencias

Cuando una clase necesita de otra para trabajar, hay dos formas de obtenerla: crearla dentro (acoplamiento rígido) o recibirla desde fuera, normalmente por el constructor. Esta segunda técnica se llama inyección de dependencias y hace el código más flexible y fácil de probar. El siguiente archivo la muestra completa y ejecutable, sin base de datos.

InyeccionDemo.java

```

1 import java.util.*;
2
3 public class InyeccionDemo {
4     public static void main(String[] args) {
5         var repo = new EstudianteRepositorio();
6         var servicio = new EstudianteServicio(repo); // se inyecta el repo
7         servicio.listar().forEach(System.out::println);
8     }
9 }
10
11 record Estudiante(int id, String nombre) {}
12
13 class EstudianteRepositorio {
14     public List<Estudiante> listarTodos() {
15         return List.of(new Estudiante(1, "Ana"), new Estudiante(2, "Luis"));
16     }
17 }
18
19 class EstudianteServicio {
20     private final EstudianteRepositorio repositorio;
21     public EstudianteServicio(EstudianteRepositorio repositorio) {
22         this.repositorio = repositorio; // recibido, no creado
23     }
24     public List<Estudiante> listar() { return repositorio.listarTodos(); }
25 }

```

Como el servicio no decide qué repositorio usa, puedes darle uno real o uno de prueba sin cambiar su código. Programar contra interfaces (sección 4.6) potencia aún más esta idea.

8.5 Ejercicios — organización

Estos ejercicios te hacen estructurar el código como en un proyecto real.

Organiza y desacopla:

1. **[Intermedio]** Reparte una pequeña aplicación en paquetes `modelo`, `servicio` y `Main`, con sus

```
import.
```

2. **[Intermedio]** Configura un `pom.xml` con la dependencia del conector de PostgreSQL y compilalo con `mvn compile`.
3. **[Avanzado]** Define una interfaz `Repositorio` y dos implementaciones; inyéctala en un servicio por el constructor.
4. **[Avanzado]** Ajusta la visibilidad de una clase para que su estado solo sea accesible mediante métodos públicos.

9 Trabajar con una base de datos (JDBC)

La mayoría de los programas necesitan guardar y recuperar información de forma permanente. Java accede a las bases de datos mediante JDBC (Java Database Connectivity), un conjunto de clases estándar del paquete `java.sql`. En esta sección conectarás con PostgreSQL y harás las operaciones básicas de forma segura.

Para ejecutar los ejemplos de esta sección

Necesitas PostgreSQL en ejecución, la base `appweb2026ppa` con la tabla `estudiantes`, y el conector `org.postgresql:postgresql` (añadido como dependencia de Maven). Los archivos compilan con solo el JDK, pero al ejecutarlos requieren ese conector.

9.1 ¿Qué es JDBC?

JDBC es la interfaz estándar de Java para comunicarse con bases de datos relacionales. Funciona con cualquier motor (PostgreSQL, MySQL, etc.) cambiando solo el conector y la cadena de conexión. Necesitas añadir el conector del motor como dependencia de Maven; para PostgreSQL es `org.postgresql:postgresql`.

9.2 La conexión

Para conectar necesitas una cadena de conexión (que indica el motor, la dirección, el puerto y la base), el usuario y la contraseña. La conexión se obtiene con `DriverManager` y conviene abrirla dentro de un `try-with-resources` para que se cierre automáticamente al terminar.

ConexionDemo.java

```

1 import java.sql.*;
2
3 public class ConexionDemo {
4     public static void main(String[] args) {
5         String cadena = "jdbc:postgresql://127.0.0.1:5432/appweb2026ppa";
6         String usuario = "postgres";
7         String clave = "P@$7Gr3$QL"; // en Java las cadenas no interpolan
8         try (Connection con =
9             DriverManager.getConnection(cadena, usuario, clave)) {
10             System.out.println("Conexion exitosa");
11         } catch (SQLException e) {
12             System.out.println("Error: " + e.getMessage());
13         }
14     }
15 }

```

9.3 Sentencias preparadas

Nunca incrustes datos en una consulta concatenando texto: es la puerta a la inyección SQL. En su lugar, usa `PreparedStatement` con marcadores `?` y asigna los valores aparte; el motor los trata como datos, nunca como código. Es una defensa de seguridad imprescindible.

InsertarDemo.java

```

1 import java.sql.*;
2
3 public class InsertarDemo {
4     public static void main(String[] args) throws SQLException {
5         String cadena = "jdbc:postgresql://127.0.0.1:5432/appweb2026ppa";
6         try (Connection con =
7             DriverManager.getConnection(cadena, "postgres", "P@$7Gr3$QL")) {
8             String sql = "INSERT INTO estudiantes (nombre, correo, carrera) "
9                 + "VALUES (?, ?, ?)";
10             try (PreparedStatement ps = con.prepareStatement(sql)) {
11                 ps.setString(1, "Ana Perez");
12                 ps.setString(2, "ana@uteq.edu.ec");
13                 ps.setString(3, "Software");
14                 ps.executeUpdate();
15                 System.out.println("Insertado");
16             }
17         }
18     }
19 }

```

9.4 Consultar y recorrer resultados

Para leer datos se ejecuta una consulta y se recorre el `ResultSet` que devuelve, fila por fila, con el método `next()`. De cada fila se obtienen las columnas por nombre. Conviene también aquí el `try-with-resources` para cerrar la sentencia y el resultado.

ListarDemo.java

```

1 import java.sql.*;
2
3 public class ListarDemo {
4     public static void main(String[] args) throws SQLException {
5         String cadena = "jdbc:postgresql://127.0.0.1:5432/appweb2026ppa";
6         String sql = "SELECT id, nombre, correo FROM estudiantes ORDER BY id";
7         try (Connection con =
8             DriverManager.getConnection(cadena, "postgres", "P0$7Gr3$QL");
9             Statement st = con.createStatement();
10            ResultSet rs = st.executeQuery(sql)) {
11             while (rs.next()) {
12                 System.out.println(rs.getInt("id") + " - " + rs.getString("nombre"));
13             }
14         }
15     }
16 }

```

9.5 Transacciones

Cuando varias operaciones deben tener éxito o fracasar como un bloque, se agrupan en una transacción. Se desactiva la confirmación automática, se ejecutan las operaciones y se confirma con **commit**; si algo falla, se deshace todo con **rollback**.

TransaccionDemo.java

```

1 import java.sql.*;
2
3 public class TransaccionDemo {
4     public static void main(String[] args) throws SQLException {
5         String cadena = "jdbc:postgresql://127.0.0.1:5432/appweb2026ppa";
6         try (Connection con =
7             DriverManager.getConnection(cadena, "postgres", "P0$7Gr3$QL")) {
8             try {
9                 con.setAutoCommit(false);
10                // ... varias operaciones con PreparedStatement ...
11                con.commit(); // todo correcto
12            } catch (SQLException e) {
13                con.rollback(); // deshacer todo
14                System.out.println("Transaccion cancelada");
15            }
16        }
17    }
18 }

```

9.6 Ejercicios — base de datos

Estos ejercicios requieren PostgreSQL en ejecución y la tabla **estudiantes**. Aplica siempre **PreparedStatement**.

Trabaja con datos persistentes:

1. **[Intermedio]** Conéctate y muestra todos los estudiantes recorriendo el `ResultSet`.
2. **[Intermedio]** Inserta un estudiante con `PreparedStatement` y comprueba cuántas filas se afectaron.
3. **[Avanzado]** Implementa una búsqueda por nombre con `LIKE` y un marcador.
4. **[Avanzado]** Escribe una transacción que inserte varios estudiantes y que, si uno falla, no deje ninguno.

10 Buenas prácticas y herramientas

Escribir Java profesional implica seguir convenciones y apoyarse en herramientas que elevan la calidad. Esta sección reúne hábitos y utilidades que conviene adoptar desde el principio, porque marcan la diferencia entre un código que solo funciona y uno que se puede mantener.

10.1 Convenciones de código

La comunidad Java sigue convenciones de nombres muy establecidas: las clases en `PascalCase` (como `EstudianteServicio`), los métodos y variables en `camelCase` (como `calcularTotal`), y las constantes en `MAYUSCULAS`. Respetarlas hace tu código inmediatamente legible para cualquier otro programador.

10.2 Pruebas con JUnit

Las pruebas automatizadas verifican que tu código hace lo que debe y que sigue funcionando tras los cambios. JUnit es el estándar en Java. El siguiente archivo es una prueba completa; acompaña a una clase `Calculadora` del proyecto y se ejecuta con `mvn test` (necesita la dependencia de JUnit 5).

CalculadoraTest.java

```
1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 class CalculadoraTest {
5     @Test
6     void suma() {
7         var c = new Calculadora();
8         assertEquals(5, c.sumar(2, 3));
9     }
10 }
```

10.3 Registro de eventos (logging)

En lugar de llenar el código de `System.out.println`, los programas serios registran sus eventos con un sistema de logging, que permite clasificar los mensajes por importancia (información, advertencia, error) y controlarlos sin tocar el código. Es la forma adecuada de dejar rastro de lo que ocurre.

10.4 Empaquetado en un .jar

Cuando tu programa está listo, se distribuye empaquetado en un archivo `.jar`, que reúne todas tus clases compiladas. Con Maven, `mvn package` lo genera en la carpeta `target`. Ese `.jar` se ejecuta con `java -jar`, y es la forma estándar de entregar un programa Java.

10.5 Control de versiones

Usar un control de versiones como Git desde el primer día te permite guardar el historial de tu trabajo, volver atrás y colaborar. Recuerda no incluir en el repositorio lo generado (la carpeta `target`) ni datos sensibles; para eso sirve el archivo `.gitignore`.

11 Proyecto integrador: gestor de estudiantes en consola

Es hora de unir todo en un programa completo que funciona en la consola: un gestor de estudiantes con un menú que permite listar, agregar y eliminar, guardando los datos en la base. El proyecto aplica records, colecciones, excepciones, JDBC y una arquitectura por capas con inyección de dependencias.

11.1 Estructura del proyecto

Separaremos las responsabilidades en capas, una práctica que escala muy bien: el modelo (los datos), el repositorio (el acceso a la base), el servicio (la lógica) y el `Main` (la interacción con la persona por consola). Cada archivo es completo y va en la carpeta de su paquete.

Capas del proyecto

```
1 src/main/java/com/uteq/
2 |- Main.java           # menu de consola
3 |- modelo/Estudiante.java # record con los datos
4 |- repositorio/EstudianteRepositorio.java # acceso JDBC
5 |- servicio/EstudianteServicio.java      # logica y validacion
```

11.2 El modelo

El modelo es un `record` que representa a un estudiante. Al ser inmutable y conciso, es perfecto para transportar datos entre capas sin código repetitivo. Va en su paquete `modelo`.

modelo/Estudiante.java

```
1 package com.uteq.modelo;
2
3 public record Estudiante(int id, String nombre,
4                          String correo, String carrera) {}
```

11.3 El repositorio (acceso a datos)

El repositorio encapsula todo el JDBC: recibe la conexión por el constructor (inyección de dependencias) y ofrece métodos para listar, guardar y eliminar. El resto del programa no sabe nada de SQL: solo llama a estos métodos.

repositorio/EstudianteRepositorio.java

```

1 package com.uteq.repositorio;
2
3 import com.uteq.modelo.Estudiante;
4 import java.sql.*;
5 import java.util.*;
6
7 public class EstudianteRepositorio {
8     private final Connection con;
9     public EstudianteRepositorio(Connection con) { this.con = con; }
10
11     public List<Estudiante> listarTodos() throws SQLException {
12         var lista = new ArrayList<Estudiante>();
13         String sql = "SELECT id, nombre, correo, carrera "
14             + "FROM estudiantes ORDER BY id";
15         try (var st = con.createStatement();
16             var rs = st.executeQuery(sql)) {
17             while (rs.next()) {
18                 lista.add(new Estudiante(
19                     rs.getInt("id"), rs.getString("nombre"),
20                     rs.getString("correo"), rs.getString("carrera")));
21             }
22         }
23         return lista;
24     }
25
26     public void guardar(Estudiante e) throws SQLException {
27         String sql = "INSERT INTO estudiantes (nombre, correo, carrera) "
28             + "VALUES (?, ?, ?)";
29         try (var ps = con.prepareStatement(sql)) {
30             ps.setString(1, e.nombre());
31             ps.setString(2, e.correo());
32             ps.setString(3, e.carrera());
33             ps.executeUpdate();
34         }
35     }
36
37     public void eliminar(int id) throws SQLException {
38         try (var ps = con.prepareStatement(
39             "DELETE FROM estudiantes WHERE id = ?")) {
40             ps.setInt(1, id);
41             ps.executeUpdate();
42         }
43     }
44 }

```

11.4 El servicio (lógica y validación)

El servicio añade la lógica de negocio: valida los datos antes de guardarlos y coordina al repositorio. Recibe el repositorio por el constructor, de modo que no depende de cómo se accede a la base.

servicio/EstudianteServicio.java

```
1 package com.uteq.servicio;
2
3 import com.uteq.modelo.Estudiante;
4 import com.uteq.repositorio.EstudianteRepositorio;
5 import java.sql.SQLException;
6 import java.util.List;
7
8 public class EstudianteServicio {
9     private final EstudianteRepositorio repo;
10    public EstudianteServicio(EstudianteRepositorio repo) {
11        this.repo = repo;
12    }
13
14    public List<Estudiante> listar() throws SQLException {
15        return repo.listarTodos();
16    }
17
18    public void registrar(String nombre, String correo, String carrera)
19        throws SQLException {
20        if (nombre.isBlank() || correo.isBlank() || carrera.isBlank()) {
21            throw new IllegalArgumentException("Todos los campos son obligatorios");
22        }
23        repo.guardar(new Estudiante(0, nombre, correo, carrera));
24    }
25
26    public void eliminar(int id) throws SQLException { repo.eliminar(id); }
27 }
```

11.5 El programa principal (menú de consola)

Finalmente, el Main arma las piezas (crea la conexión, el repositorio y el servicio) y presenta un menú por consola con **Scanner**. Observa cómo la inyección de dependencias conecta las capas en un solo lugar.

Main.java

```

1 package com.uteq;
2
3 import com.uteq.repositorio.EstudianteRepositorio;
4 import com.uteq.servicio.EstudianteServicio;
5 import java.sql.*;
6 import java.util.Scanner;
7
8 public class Main {
9     public static void main(String[] args) throws SQLException {
10         String cadena = "jdbc:postgresql://127.0.0.1:5432/appweb2026ppa";
11         try (Connection con = DriverManager.getConnection(
12             cadena, "postgres", "P0$7Gr3$QL")) {
13
14             var repo = new EstudianteRepositorio(con);
15             var servicio = new EstudianteServicio(repo);    // inyeccion
16             var sc = new Scanner(System.in);
17
18             int op;
19             do {
20                 System.out.println("\n1) Listar 2) Agregar 3) Eliminar 0) Salir");
21                 op = Integer.parseInt(sc.nextLine());
22                 switch (op) {
23                     case 1 -> servicio.listar().forEach(System.out::println);
24                     case 2 -> {
25                         System.out.print("Nombre: "); var n = sc.nextLine();
26                         System.out.print("Correo: "); var c = sc.nextLine();
27                         System.out.print("Carrera: "); var ca = sc.nextLine();
28                         servicio.registrar(n, c, ca);
29                     }
30                     case 3 -> {
31                         System.out.print("Id a eliminar: ");
32                         servicio.eliminar(Integer.parseInt(sc.nextLine()));
33                     }
34                 }
35             } while (op != 0);
36         }
37     }
38 }

```

Con estas cuatro clases tienes un programa completo y bien estructurado: el modelo transporta datos, el repositorio habla con la base, el servicio valida y el Main interactúa con la persona. Cada pieza se entiende y se prueba por separado.

11.6 Ejercicio integrador

Amplía el proyecto para consolidar todo lo aprendido.

Lleva el proyecto más lejos:

1. **[Avanzado]** Añade la operación **actualizar** (modificar el correo de un estudiante por su id).
2. **[Avanzado]** Añade una búsqueda por carrera que muestre solo los estudiantes de esa carrera.
3. **[Avanzado]** Extrae una interfaz **Repositorio** e implementa una versión en memoria para probar

el servicio sin base de datos.

4. **[Avanzado]** Escribe pruebas con JUnit para la validación del servicio usando esa versión en memoria.
5. **[Avanzado]** Reemplaza los `System.out.println` por un sistema de logging.

12 Banco de ejercicios por nivel

Este banco reúne ejercicios adicionales para practicar de forma intensiva, agrupados por nivel. Resuélvelos en tu entorno y, cuando puedas, escribe pruebas que comprueben tu solución.

12.1 Nivel básico

Refuerzan la sintaxis, los tipos, los arreglos y los métodos.

1. Determina si un número es primo.
2. Cuenta cuántas vocales tiene una cadena.
3. Calcula el factorial de un número con un método recursivo.
4. Encuentra el mayor y el menor de un arreglo.
5. Convierte una temperatura de Celsius a Fahrenheit en un método.
6. Comprueba si una palabra es un palíndromo.
7. Muestra los números del 1 al 100 múltiplos de 3 o de 5.

12.2 Nivel intermedio

Combinan orientación a objetos, colecciones y excepciones.

1. Clase `Libro` y una lista de libros con alta, baja y listado.
2. Cuenta la frecuencia de cada palabra de un texto con un `Map`.
3. Jerarquía de figuras con una interfaz y cálculo de áreas (polimorfismo).
4. Conversor de unidades con un `enum` de unidades y `switch`.
5. Método genérico que devuelva el máximo de una lista de elementos comparables.
6. Validador que lance excepciones propias para datos inválidos.

12.3 Nivel avanzado

Exigen diseño, programación funcional y trabajo con datos.

1. Procesa una lista de `Estudiante` con streams: agrupa por carrera y cuenta.

2. Repositorio con interfaz y dos implementaciones (memoria y JDBC).
3. Gestor de tareas en consola con prioridades, ordenado con `Comparator`.
4. Lee un archivo de texto con `try-with-resources` y resume su contenido.
5. Suite de pruebas JUnit para la lógica de un servicio.
6. Transacción JDBC que mueva un estudiante de una carrera a otra de forma atómica.

12.4 Proyectos propuestos

Para integrar todo en algo mayor, elige un proyecto y desarróllalo por etapas, con su arquitectura por capas y sus pruebas.

1. Agenda de contactos en consola con persistencia en base de datos.
2. Gestor de calificaciones con reportes por estudiante y por asignatura.
3. Inventario de productos con altas, bajas, búsquedas y control de stock.
4. Biblioteca con préstamos y devoluciones, validando disponibilidad en transacciones.

13 Soluciones seleccionadas

Aquí tienes soluciones comentadas de algunos ejercicios del banco para contrastar tu enfoque. Cada una es un archivo completo y ejecutable. Casi siempre hay más de una solución válida; lo importante es que sea correcta, clara y bien estructurada.

13.1 Número primo (básico 12.1.1)

Un número es primo si solo es divisible por 1 y por sí mismo. Basta con probar divisores hasta su raíz cuadrada para ser eficientes.

Primo.java

```
1 public class Primo {
2     static boolean esPrimo(int n) {
3         if (n < 2) return false;
4         for (int i = 2; i * i <= n; i++) {
5             if (n % i == 0) return false;
6         }
7         return true;
8     }
9     public static void main(String[] args) {
10        System.out.println(esPrimo(7)); // true
11        System.out.println(esPrimo(8)); // false
12    }
13 }
```

13.2 Frecuencia de palabras (intermedio 12.2.2)

Usamos un `Map` para contar. Por cada palabra, `getOrDefault` devuelve el conteo actual (o 0) y le sumamos uno.

ContarPalabras.java

```
1 import java.util.*;
2
3 public class ContarPalabras {
4     public static void main(String[] args) {
5         String texto = "ana luis ana eva luis ana";
6         Map<String, Integer> conteo = new HashMap<>();
7         for (String palabra : texto.split(" ")) {
8             conteo.put(palabra, conteo.getOrDefault(palabra, 0) + 1);
9         }
10        System.out.println(conteo);    // {ana=3, luis=2, eva=1}
11    }
12 }
```

13.3 Agrupar con streams (avanzado 12.3.1)

Con `Collectors.groupingBy` agrupamos los estudiantes por carrera en un mapa, y con `counting()` obtenemos cuántos hay en cada una, todo en una expresión.

AgruparPorCarrera.java

```
1 import java.util.*;
2 import java.util.stream.*;
3
4 public class AgruparPorCarrera {
5     record Estudiante(int id, String nombre, String carrera) {}
6
7     public static void main(String[] args) {
8         List<Estudiante> estudiantes = List.of(
9             new Estudiante(1, "Ana", "Software"),
10            new Estudiante(2, "Luis", "Telematica"),
11            new Estudiante(3, "Sara", "Software"));
12        Map<String, Long> porCarrera = estudiantes.stream()
13            .collect(Collectors.groupingBy(
14                Estudiante::carrera, Collectors.counting()));
15        System.out.println(porCarrera);
16    }
17 }
```

14 Glosario de términos

Este glosario reúne los términos y siglas que aparecen en la guía, para consulta rápida durante el estudio.

Término	Significado
JDK	Java Development Kit (compilador, máquina virtual y bibliotecas).
JVM	Java Virtual Machine (ejecuta el bytecode).
Bytecode	Formato intermedio al que se compila Java.
LTS	Long-Term Support (versión con soporte prolongado).
POO	Programación Orientada a Objetos.
Clase / Objeto	Molde / instancia concreta creada con new.
Interfaz	Contrato de métodos que una clase implementa.
record	Clase concisa e inmutable para transportar datos.
enum	Conjunto fijo de valores posibles.
Genéricos	Tipos parametrizados con <> para seguridad de tipos.
Colección	Estructura de datos (List, Set, Map).
Excepción	Objeto que representa un error en ejecución.
Lambda	Función anónima breve.
Stream	Flujo para procesar colecciones declarativamente.
Optional	Envoltorio para un valor que puede faltar.
Maven	Gestor de dependencias y construcción.
JDBC	Java Database Connectivity (acceso a bases de datos).
JUnit	Marco de pruebas automatizadas de Java.

15 Recursos y referencias

Para seguir aprendiendo y consultar dudas, estos son los recursos de referencia más fiables del ecosistema Java.

- Documentación oficial de Java: docs.oracle.com/en/java

- Tutoriales oficiales (The Java Tutorials): **[dev.java/learn](https://docs.oracle.com/javase/7/docs/tutorial/index.html)**
- Distribución abierta del JDK (Temurin): **adoptium.net**
- Maven: **maven.apache.org**
- JUnit 5: **junit.org/junit5**
- PostgreSQL: **[postgresql.org/docs](https://www.postgresql.org/docs)**

Con esta guía y la práctica constante de los ejercicios, tendrás una base de Java sólida y bien estructurada, lista para abordar con confianza proyectos cada vez más ambiciosos.